Requisitos de software

1. O que são Requisitos de Software?

Requisitos de software são descrições detalhadas das necessidades, expectativas e restrições de um sistema a ser desenvolvido. Eles garantem o alinhamento entre o que os *stakeholders* (clientes, usuários, investidores) esperam e o que os desenvolvedores irão entregar.

- **Por que são importantes?** Eles servem como base para todas as fases do desenvolvimento (design, código, testes). Um bom levantamento reduz riscos de retrabalho, evita ambiguidades e define o escopo do projeto.

2. Tipos de Requisitos

Os requisitos dividem-se em duas categorias principais:

- **Requisitos Funcionais (RF):** Definem o que o sistema *deve fazer*. São as funcionalidades diretas.
  - *Exemplos:* "O sistema deve cadastrar médicos", "O sistema deve permitir a emissão de histórico escolar", "O cliente pode consultar seus dados".
  - *Níveis:* Podem ser divididos em Requisitos de Negócio, Requisitos de Usuário e Requisitos de Sistema/Técnicos.
- **Requisitos Não Funcionais (RNF):** Definem *como* o sistema deve se comportar (seus atributos de qualidade e restrições).
  - *Exemplos:* "O sistema deve responder em até 2 segundos (Desempenho)", "O sistema deve ser feito em Python (Restrição tecnológica)", "O histórico deve ser impresso em PDF".
  - *Classificação clássica:* * **Produto:** Usabilidade, eficiência, confiabilidade, desempenho.
    - **Organizacionais:** Padrões da empresa, políticas de entrega.
    - **Externos:** Legislação (ex: LGPD), interoperabilidade, ética.

Tabela: RF x RNF

Característica	Requisitos Funcionais (RF)	Requisitos Não Funcionais (RNF)
Definição	Especificam as funcionalidades, ações e comportamentos do sistema.	Especificam as qualidades, atributos e restrições técnicas do sistema.
Pergunta que responde	"O que o sistema *deve fazer*?"	"Como / quão bem o sistema *deve funcionar*?"
Impacto no funcionamento	Sem eles, o sistema não atende ao seu propósito básico (não realiza a tarefa).	Sem eles, o sistema até pode funcionar, mas com baixa qualidade, insegurança ou lentidão.
Como são medidos?	Testes de funcionalidade (Ex: A ação aconteceu com sucesso? Sim/Não).	Critérios mensuráveis (Ex: O tempo de resposta foi menor que 2 segundos?).
Foco principal	Regras de negócio, entrada de dados, processamento e saída de informações.	Desempenho, usabilidade, segurança, confiabilidade, portabilidade e legislação.
Exemplos Práticos	- Permitir o cadastro de médicos. - Emitir um relatório em PDF. - Calcular a média de notas de um aluno.	- O sistema deve carregar em até 3 segundos. - As senhas devem ser salvas com criptografia. - A interface deve rodar em Android e iOS.

3. Processos da análise de requisitos

O levantamento de requisitos segue um ciclo com 5 etapas fundamentais:

1. **Levantamento (Elicitação):** Envolve identificar os *stakeholders* e entender o domínio do negócio para coletar as reais necessidades.
2. **Análise e Negociação:** Determinar se os requisitos são viáveis, se estão completos ou se há conflitos entre eles (ex: um requisito exige alta segurança, mas outro exige acesso sem senha).
3. **Documentação (Especificação):** Registro claro dos requisitos. Pode ser feito de forma tradicional (Documento de Requisitos de Software - SRS, Casos de Uso) ou ágil (Histórias de Usuário).

4. **Validação:** Confirmação de que os requisitos documentados realmente representam o que o cliente quer (através de revisões ou protótipos).
5. **Gerenciamento:** Controle de mudanças. Como as necessidades mudam com o tempo, toda alteração deve ser registrada, avaliada e aprovada.

4. Técnicas de Levantamento (Elicitação)

Para descobrir o que o sistema precisa ter, usa-se diversas abordagens:

- **Entrevistas:** Conversas diretas com stakeholders.
- **Questionários:** Útil para coletar dados de muitos usuários rapidamente.
- **Workshop / Brainstorming:** Reuniões criativas em grupo para geração de ideias.
- **Observação (Etnografia):** Acompanhar o usuário trabalhando no dia a dia para entender suas dores reais.
- **Prototipagem:** Criar telas falsas para o usuário validar visualmente.
- **JAD (Joint Application Design):** Sessões intensivas e estruturadas envolvendo clientes e desenvolvedores.

5. Priorização de Requisitos

Nem tudo pode ser feito ao mesmo tempo. A técnica mais famosa para priorizar o que desenvolver primeiro é o **MoSCoW**:

- **M**ust have (Obrigatório / Crítico)
- **S**hould have (Importante, mas não impede o lançamento)
- **C**ould have (Desejável / "Seria bom ter")
- **W**on't have (Não será feito neste momento)

6. Abordagem Tradicional vs. Ágil

- **Tradicional (Cascata):** Exige um documento gigante e formalizado (Especificação de Requisitos de Software - ERS) antes de escrever qualquer código. Foca em *Casos de Uso*.

- **Ágil (Scrum / Kanban):** Os requisitos são dinâmicos. Em vez de documentos longos, usam-se **Histórias de Usuário** (ex: *"Como [tipo de usuário], eu quero [ação] para que [motivo/benefício]"*) organizadas em um *Backlog*.

7. Problemas Comuns na Análise de Requisitos

- Falha na comunicação entre o cliente (linguagem de negócios) e o analista (linguagem técnica).
- Falta de conhecimento sobre o domínio (ex: programar um app bancário sem entender regras do Banco Central).
- Mudança constante de escopo.

Links usados:

https://www.devlingo.com.br/termos/software-requirements-specification

https://www.batebyte.pr.gov.br/Pagina/Requisitos-funcionais-e-nao-funcionais-duas-faces-da-moeda-aplicaveis-engenharia-de-software

https://www.youtube.com/watch?v=VcOeM2AD8Yk

https://www.devmedia.com.br/introducao-a-requisitos-de-software/29580